Practical No. 11: *Implement program to use PCA technique to reduce the number of features while retaining important information.

X. Conclusion

PCA is an effective technique to reduce data dimensions while keeping most variance. In this practical, the dataset was reduced to two principal components, clearly showing class separation. PCA helps in visualization, removes redundancy, and improves machine learning model performance.

XI. Practical Related Questions

1. What is PCA and why is it used?

PCA (Principal Component Analysis) is a statistical technique used to reduce the number of features in a dataset while retaining important information.
It is used to:

- Reduce dimensionality
- Remove redundancy
- Improve model performance
- Help in data visualization

---

2. What are eigenvalues and eigenvectors and how do they relate to PCA?

- **Eigenvectors** represent the directions (principal components) of maximum variance in data.
- **Eigenvalues** represent the importance (amount of variance) of those directions.

In PCA:

- Eigenvectors define new feature axes
- Eigenvalues help select the most important components

---

3. Why do we standardize data before applying PCA?

Data is standardized to:

- Ensure all features are on the same scale
- Prevent features with large values from dominating
- Improve accuracy of PCA results

4. How is PCA different from LDA?

PCA	LDA
Unsupervised learning	Supervised learning
Maximizes variance	Maximizes class separation
Does not use labels	Uses class labels
Used for feature reduction	Used for classification

---

5. What are the limitations of PCA?

- It may lose important information
- Not suitable for non-linear data
- Components are difficult to interpret
- Sensitive to scaling

---

6. How does PCA handle correlated features in a dataset?

PCA converts correlated features into **uncorrelated principal components**, removing redundancy and improving efficiency.

XII. Exercise

Answer:

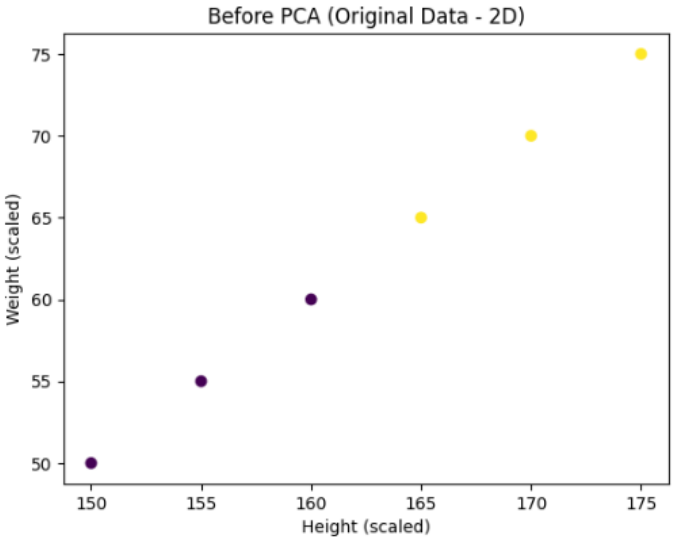
To implement PCA and evaluate its effectiveness using a classification algorithm:

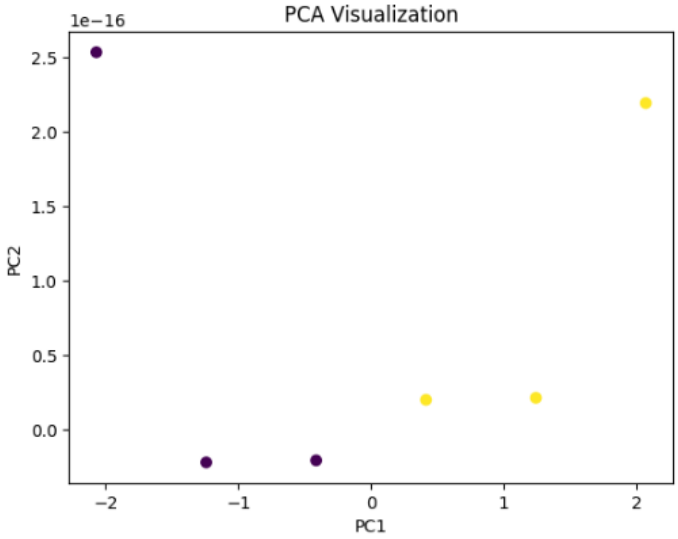
Steps Performed:

1. Loaded dataset
2. Standardized the data
3. Applied PCA to reduce dimensions
4. Visualized data in 2D
5. Applied classification algorithm (KNN)
6. Evaluated model accuracy

```python
# Import Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load dataset (Height, Weight, Label)
data = np.array([
    [150, 50, 0],
    [155, 55, 0],
    [160, 60, 0],
    [165, 65, 1],
    [170, 70, 1],
    [175, 75, 1]
])

# Separate features (X) and label (y)
X = data[:, :-1]   # Height, Weight
y = data[:, -1]    # Label

# BEFORE PCA (Original Data Visualization)
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel("Height (scaled)")
plt.ylabel("Weight (scaled)")
plt.title("Before PCA (Original Data - 2D)")
plt.show()

# 2. Standardize the data
X = StandardScaler().fit_transform(X)

# 3. Apply PCA (reduce to 2D)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# 4. Visualize data
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Visualization")
plt.show()

# 5. Apply KNN
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_pca, y)

# 6. Evaluate accuracy
y_pred = model.predict(X_pca)
print("Accuracy:", accuracy_score(y, y_pred))
```

Output





- **Original Data (2D)** – Height & Weight are correlated and plotted in 2D.
- **Standardization** – Scales data so both features are equally important.
- **PCA Transformation** – Converts Height & Weight into **principal components (PC1, PC2)**.
    - PC1 captures **most variation**
    - PC2 captures **very little variation**

- **Visualization** – Points are mostly along **PC1**, showing 2D → effectively 1D.
- **KNN Classification** – Using PC1 & PC2, model separates Fit (1) and Not Fit (0) perfectly.
- **Accuracy** – 1.0 → PCA + KNN works well even after reducing dimensions.

Conclusion

"PCA reduces correlated features into a smaller number of principal components, keeping important information, and allows KNN to classify data accurately."